Project 01

Launch Your First Production-Ready VPC

By: danuja

Introduction

This project focuses on designing and deploying a production-ready Virtual Private Cloud (VPC) on AWS using a secure and highly available architecture. The goal is to build a multi-AZ network environment that supports real-world application deployments while following industry best practices.

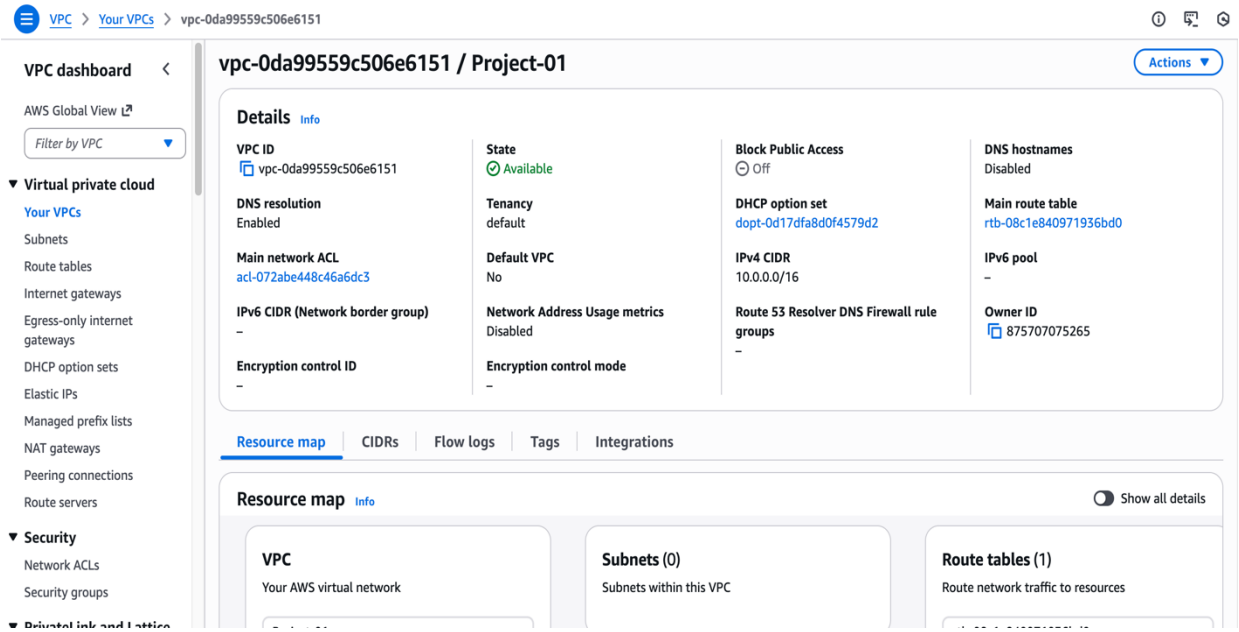
The VPC will include two public and two private subnets distributed across separate Availability Zones to ensure high availability and fault tolerance. Public subnets will host internet-facing resources through an Internet Gateway, while private subnets remain isolated and access the internet securely using a NAT Gateway. Custom route tables will be configured to properly direct traffic between the public and private layers.

To strengthen monitoring and security visibility, VPC Flow Logs will be enabled and delivered to both CloudWatch Logs and Amazon S3. Finally, test EC2 instances will be launched in each subnet to validate SSH, HTTP, and internal connectivity.
Overall, this project demonstrates the foundational AWS networking skills required to build scalable, secure, and production-grade cloud environments.

Project Tasks

1. **Build a multi-AZ VPC with:**
    - 2 Public Subnets
    - 2 Private Subnets
    - Spread across 2 Availability Zones
2. **Create a NAT Gateway** in the public subnet.
3. **Configure Route Tables** for public/private segregation.
4. **Attach an Internet Gateway** for public access.
5. **Enable VPC Flow Logs** and send them to:
    - CloudWatch Logs
    - S3 Bucket
6. **Launch a test EC2 instance** in each subnet.
7. **Validate connectivity** using:
    - SSH
    - HTTP

Task 1 – Create the VPC



Task 2 – Create 2 Public + 2 Private Subnets (across 2 AZs)

Create Public Subnets

1. Go to **Subnets → Create subnet**
2. Select **VPC**
3. Add **Subnet 1 (Public)**:
   - Name: `Public-Subnet-a`
   - AZ: `set a local`
   - CIDR: `10.0.1.0/24`
4. Click **Add new subnet**:
   - Name: `Public-Subnet-b`
   - AZ: : `set a local`
   - CIDR: `10.0.2.0/24`
5. Click **Create subnet**.

Now mark them as public (auto-assign public IP):

1. Select `Public-Subnet-A` → **Actions → Edit subnet settings**
   - Tick **Enable auto-assign public IPv4 address**
2. Do the same for `Public-Subnet-B`.

VPC > Subnets > subnet-0a5b30bae4af4c79a > Edit subnet settings

Edit subnet settings Info

Subnet

Subnet ID
subnet-0a5b30bae4af4c79a

Name
project01_public_subnet-B

Auto-assign IP settings Info

Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

- [x] Enable auto-assign public IPv4 address Info
- [ ] Enable auto-assign customer-owned IPv4 address Info
  Option disabled because no customer owned pools found.

Resource-based name (RBN) settings Info

Specify the hostname type for EC2 instances in this subnet and optional RBN DNS query settings.

- [ ] Enable resource name DNS A record on launch Info
- [ ] Enable resource name DNS AAAA record on launch Info

Hostname type Info

- ( ) Resource name
- (•) IP name

DNS64 settings

Enable DNS64 to allow IPv6-only services in Amazon VPC to communicate with IPv4-only services and networks.

---

subnet-0a5b30bae4af4c79a / project01_public_subnet-B

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Details

Subnet ID
subnet-0a5b30bae4af4c79a

IPv4 CIDR
10.0.2.0/24

Availability Zone
eun1-az2 (eu-north-1b)

Network ACL
acl-072abe448c46a6dc3

Auto-assign customer-owned IPv4 address

Subnet ARN
arn:aws:ec2:eu-north-1:875707075265:subnet/subnet-0a5b30bae4af4c79a

Available IPv4 addresses
251

Network border group
eu-north-1

Default subnet
No

State
Available

IPv6 CIDR
–

VPC
vpc-0da99559c506e6151 | Project-01

Auto-assign public IPv4 address
Yes

Outpost ID
–

Block Public Access
Off

IPv6 CIDR association ID
–

Route table
rtb-08c1e840971936bd0

Auto-assign IPv6 address
No

IPv4 CIDR reservations
–

Subnets (1/3) Info

Last updated 2 minutes ago | Actions | Create subnet

Find subnets by attribute or tag

1

Name	Subnet ID	State	VPC	Block Public...	IPv4 C
project01_public_subnet-A	subnet-01f950c231075cbed	Available	vpc-0da99559c506e6151 | Proj...	Off	10.0.1.
–	subnet-04ae4e5b67c4c6a8f	Available	vpc-0e32170621a153b4e	Off	172.31
project01_public_subnet-B	subnet-0a5b30bae4af4c79a	Available	vpc-0da99559c506e6151 | Proj...	Off	10.0.2.

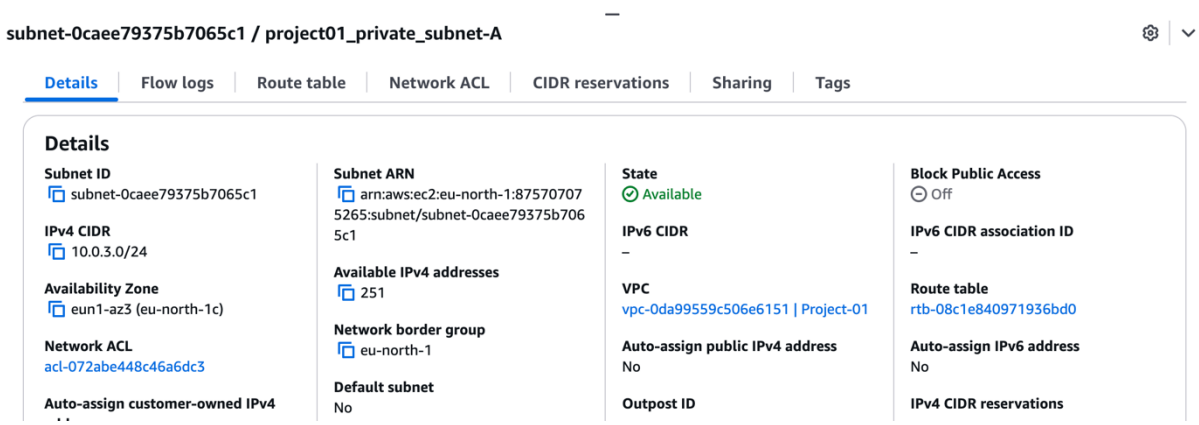
---

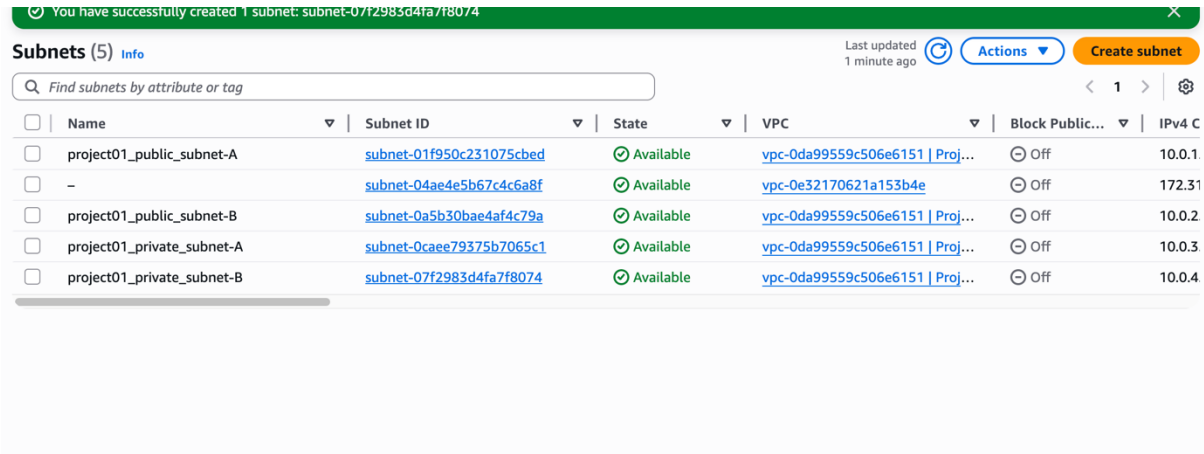
subnet-0a5b30bae4af4c79a / project01_public_subnet-B

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Create Private Subnets

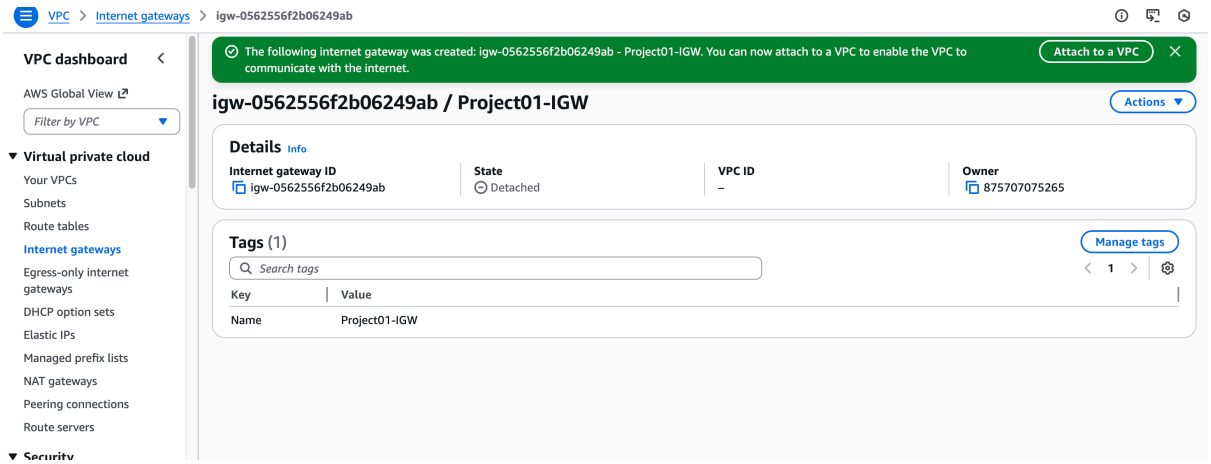
1. **Create subnet** again:
   - VPC: `ProdVPC`
   - Name: `Private-Subnet-A`
   - AZ: `eu-west-2a`
   - CIDR: `10.0.3.0/24`
2. Add another:
   - Name: `Private-Subnet-B`
   - AZ: `eu-west-2b`
   - CIDR: `10.0.4.0/24`
3. Click **Create subnet**.



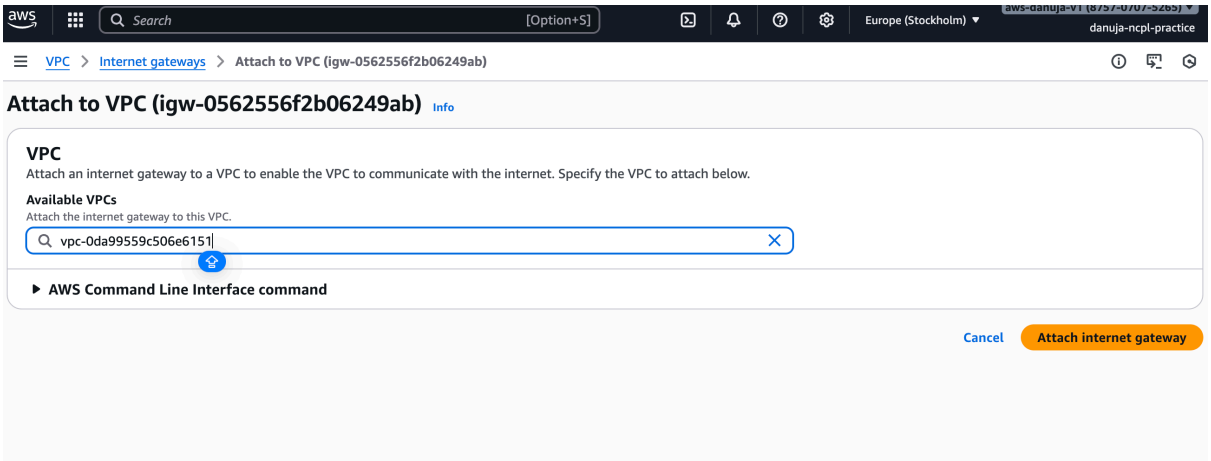


Task 3 – Create and Attach an Internet Gateway

1. Go to **Internet Gateways → Create internet gateway**
2. Name: `Prod-IGW`
3. Create.
4. Select it → **Actions → Attach to VPC** → choose `ProdVPC` → Attach.

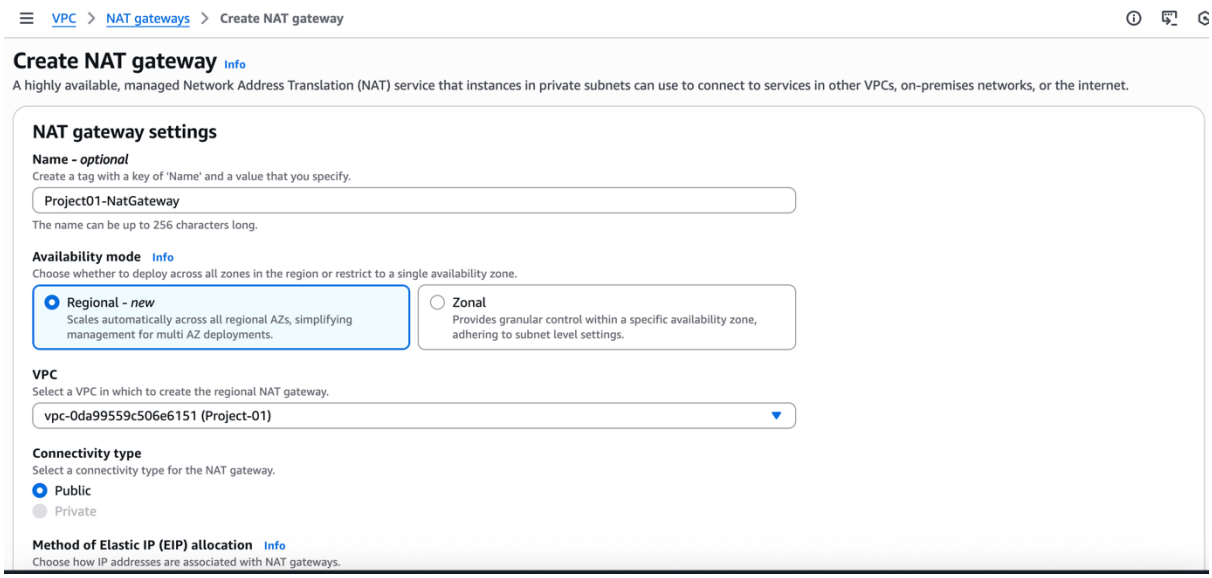


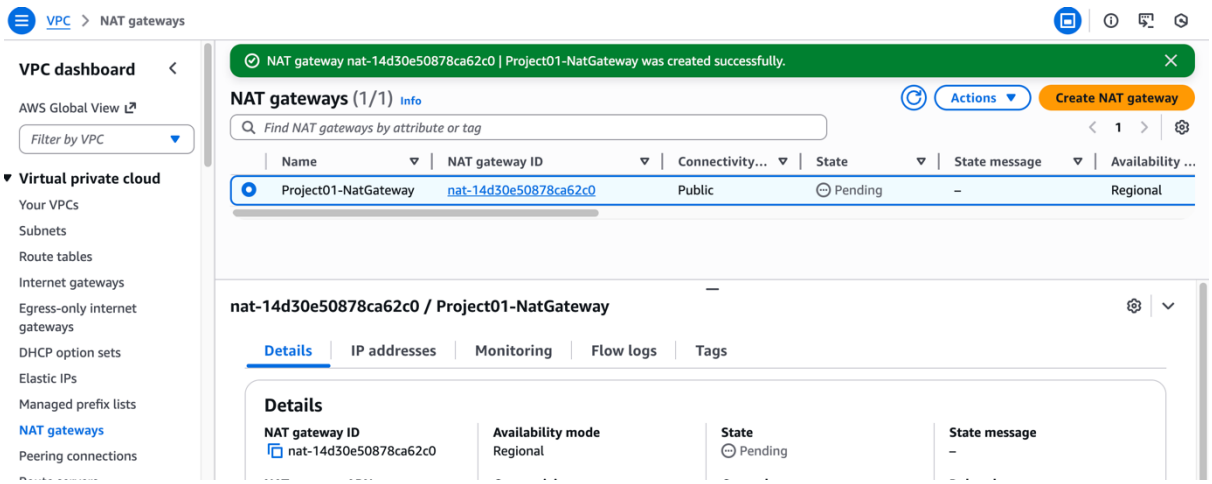
Attach to VPC



Task 4 – Create a NAT Gateway (in Public Subnet)

1. Go to **NAT Gateways → Create NAT Gateway**
2. Settings:
   - Name: `Prod-NATGW`
   - Subnet: `Public-Subnet-A` (choose one public subnet)
   - Connectivity type: `Public`
3. Click **Create NAT Gateway** and wait until status is **Available**.

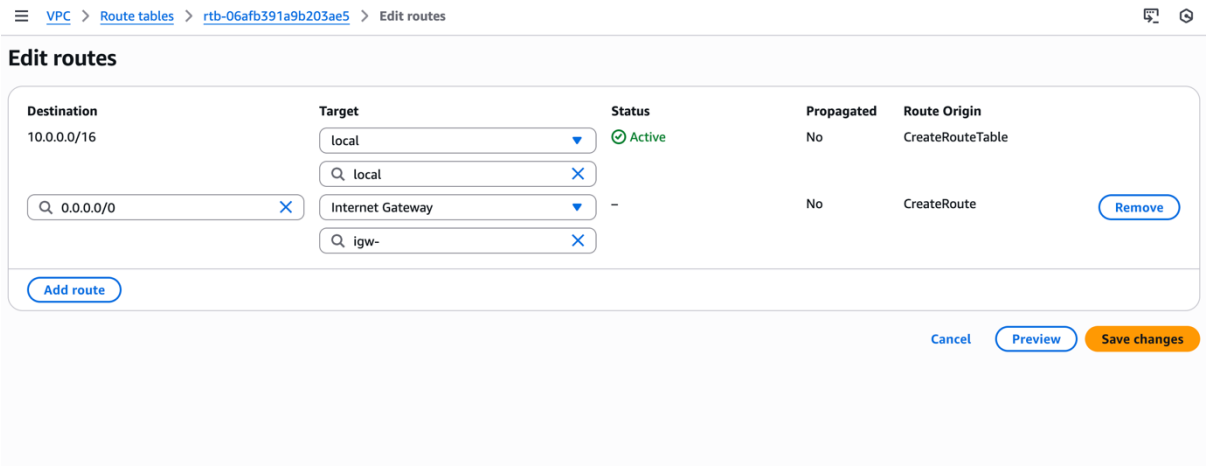




Task 5 – Create Route Tables (Public + Private)

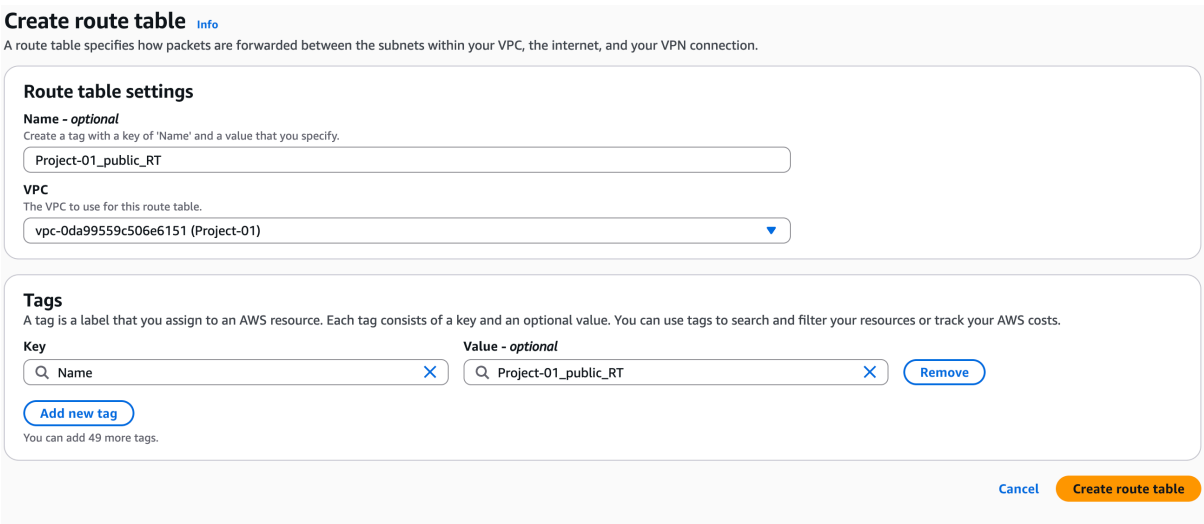
Public Route Table

1. Go to **Route Tables → Create route table**
2. Name: `Public-RT`
3. VPC: `ProdVPC`
4. Create.



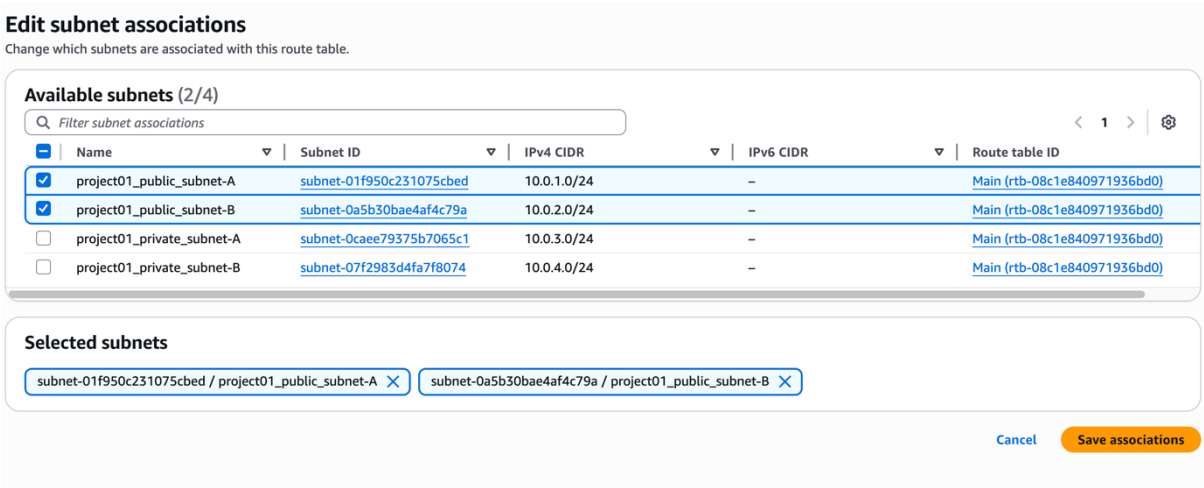
Now add IGW route:

1. Select `Public-RT` → **Routes** tab → **Edit routes**
2. Add route:
   - Destination: `0.0.0.0/0`
   - Target: **Internet Gateway** → select `Prod-IGW`
3. Save routes.



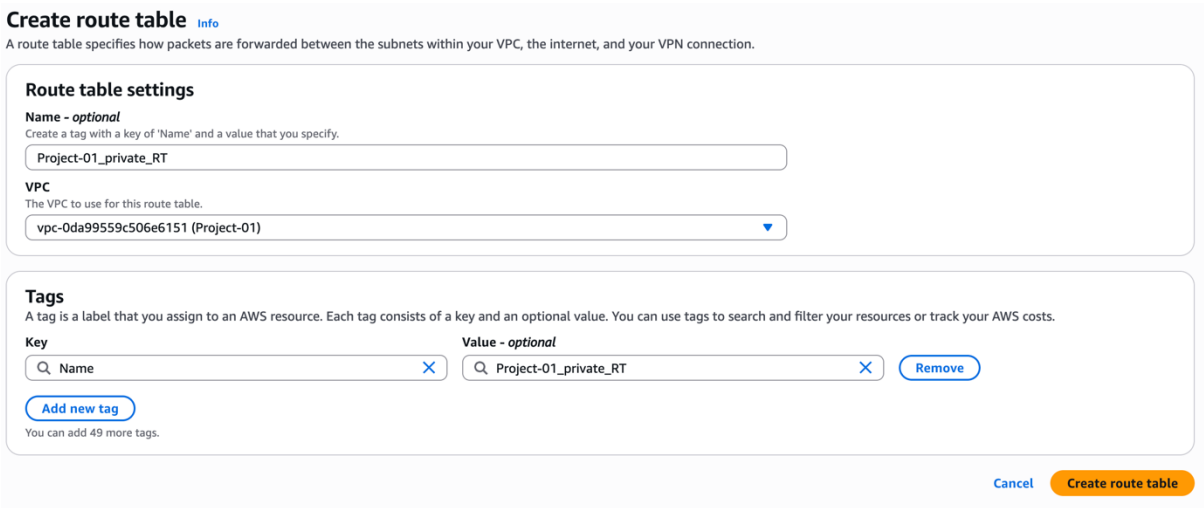
Associate to **public subnets**:

1. In `Public-RT` → **Subnet associations** → **Edit subnet associations**
2. Tick:
   - `Public-Subnet-A`
   - `Public-Subnet-B`
3. Save.



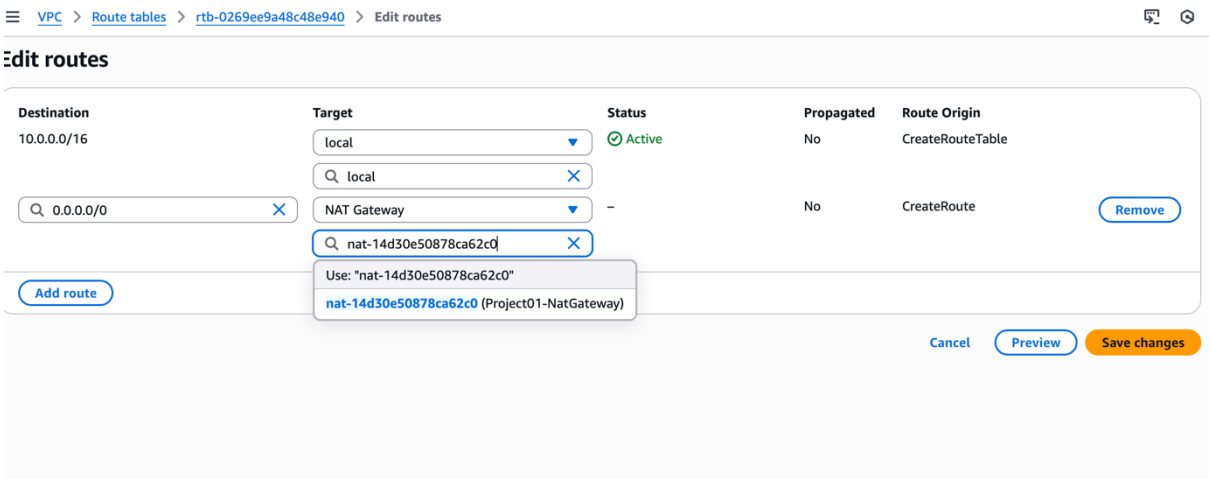
Private Route Table

1. **Create route table** again:
   - Name: `Private-RT`
   - VPC: `ProdVPC`
2. Create.



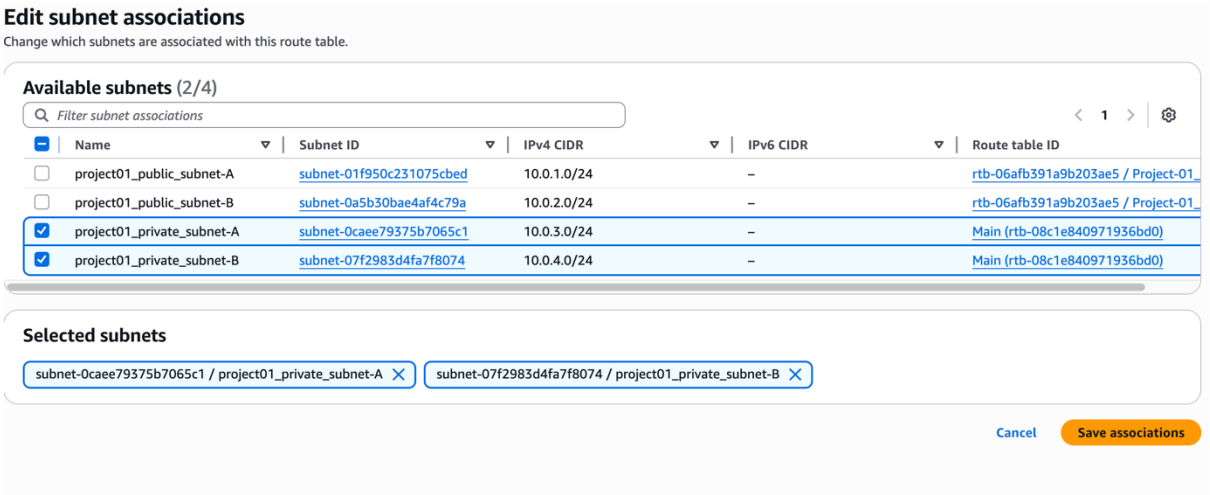
NAT route:

1. Select `Private-RT` → **Routes** → **Edit routes**
2. Add:
   - Destination: `0.0.0.0/0`
   - Target: **NAT Gateway** → `Prod-NATGW`
3. Save routes.



Associate to **private subnets**:

1. `Private-RT` → **Subnet associations** → **Edit subnet associations**
2. Tick:
   - `Private-Subnet-A`
   - `Private-Subnet-B`
3. Save.



Task 6 – Enable VPC Flow Logs (CloudWatch + S3)

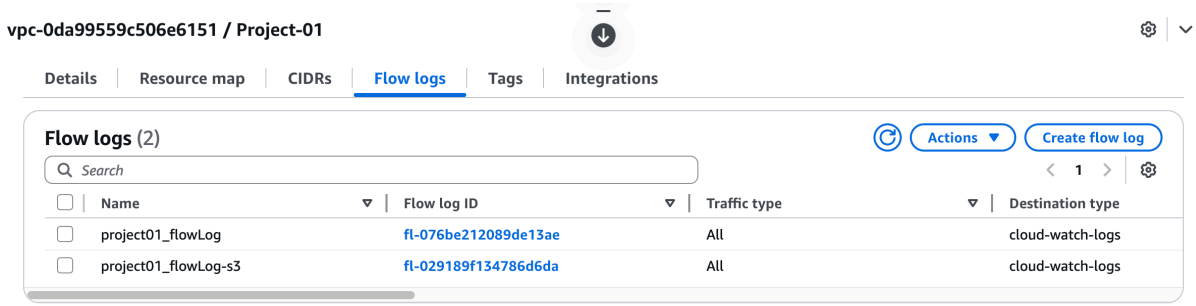
You'll create **two flow logs**:
one to **CloudWatch**, one to **S3**.

6.1 Flow Logs → CloudWatch

1. Go to **VPC → Your VPCs**
2. Select `ProdVPC` → **Flow logs** tab → **Create flow log**
3. Filter: `All`
4. Destination: **Send to CloudWatch Logs**
5. Choose/create:
   - Log group: e.g. `/aws/vpc/ProdVPC-FlowLogs`
   - IAM role: click **Set up permissions** if it suggests; accept default.
6. Create flow log.

6.2 Flow Logs → S3

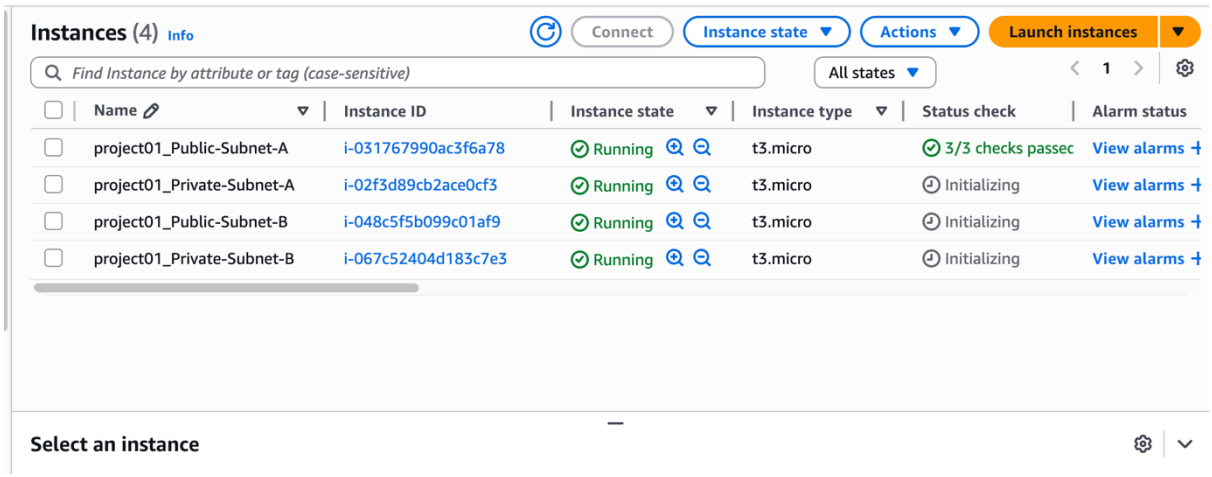
1. Again **Create flow log**
2. Filter: `All`
3. Destination: **Send to an S3 bucket**
4. Choose an existing S3 bucket or create one: `prod-vpc-flowlogs-<random>`
5. Leave default format → Create.



Task 8 – Launch EC2 Instances in Each Subnet

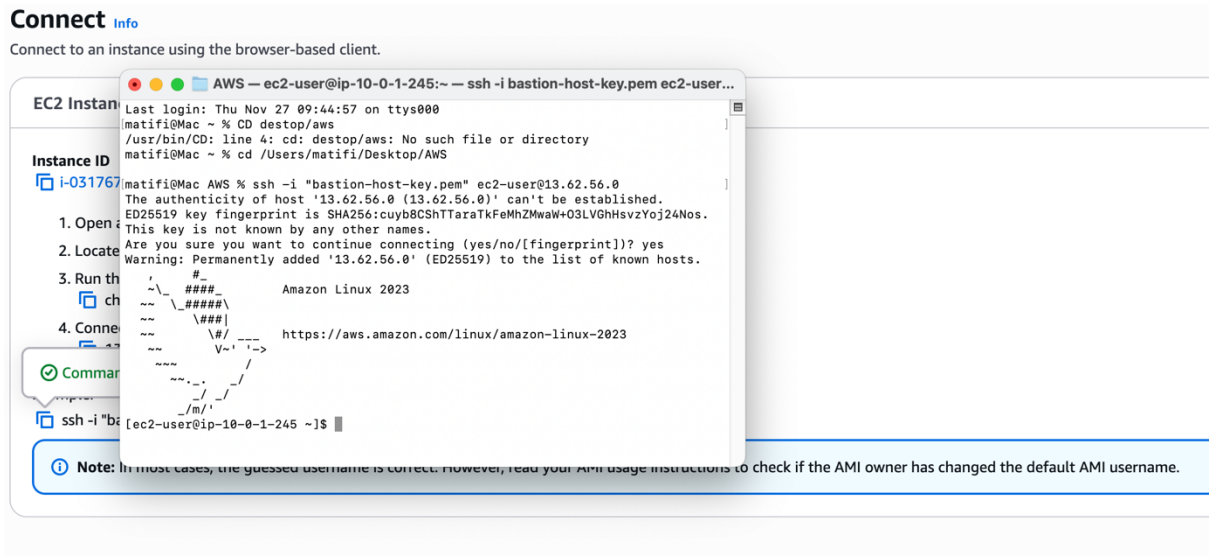
You need **4 EC2 instances**:

- 1 in `Public-Subnet-A`
- 1 in `Public-Subnet-B`
- 1 in `Private-Subnet-A`
- 1 in `Private-Subnet-B`



Task 9 – Validate Connectivity in public ec2 (SSH + HTTP)

Ssh in to project01_Public-Subnet-A



Test HTTP to Public EC2

```
31.96.20.67 - - [01/Dec/2025 20:48:34] code 404, message File not found
31.96.20.67 - - [01/Dec/2025 20:48:34] "GET /favicon.ico HTTP/1.1" 404 -
93.174.93.12 - - [01/Dec/2025 20:48:51] code 400, message Bad request version
´nãY»bhlÿ(=':©\\x82ÙoÈ¢×\\x93\\x98´ï\\x80å¹\\x90\\x00(À")
93.174.93.12 - - [01/Dec/2025 20:48:51] "\x16\x03\x02\x01o\x01\x00\x01k\x03\x0
ÅÅ\x1a#÷:NßÂ´\x82/ÿ\x09T\x9f§Äy°hÆ\x13\x8c¤\x1c="á\x1a\x98 \x84´,\x85¯nãY»bhlÿ
:©\x82ÙoÈ¢×\x93\x98´ï\x80å¹\x90\x00(À" 400 -
```

Directory listing for /

- .bash_logout
- .bash_profile
- .bashrc
- .ssh/

Connection the private EC2

1. Copy the key file content
2. Create a new file in the public ec2 end paste it

AWS — ec2-user@ip-10-0-3-24:~ — ssh -i bastion-host-key.pem ec2-user...


```
MIIEpAIBAAKCAQEAveZIuiRTTKptHNFMaKKXsjzaas/sJrT2dTwMfh0eF86s2C+N
mS7hZB+qrvtAOwmPjlv/ogRJUUSph2k9Icab6UFv5B0Fv6n5pELqn7JRrVW0y36x
fPKt1JYzA96cfzAl0/cimrrcXbEKNd5cYX8DuwV2SkchbII1/i344u8TOLumLIrC
Ngk/zNy4hlJTb0c9DjQUZ3y8wdodJuJVlwQLz0SxzsiiM3A7OVDkRsjCsC0bn88s
3cAlr7EtM8bdcc44cTsawF/uGoxqj7rrZKv5fVw5cajx5i99EmCZVS25TQ/sBwz+
Qzr0FbmfK9p8/ckQQVnVBr6osew6bhj4vA96rwIDAQABAoIBAHYDXr8PRBUo9JYs
G+gK42sJFlO6029alq9Umg7l2YXCEvhm1QCVeczXQSNtUQVfc69szPjK4SV4HkLg
y/TpkIrs97QztRbfcxz52QVVtBycXrb4DSUxS4YJsbp95ZPFQDzyILc32SsAku/X
OGH+pTrcsh6HBeaBJTzJJRsiqDdgkfoYwc6AY7Dp52lgmRmsvUsH/Ce3uf0mpfsC
IwqQ/Y3mX8VNtqDjXul/93z4tDkDHUAHyCimCAXka+n7c1rh3jqYCL7BZGgebGxH
7duVPm5dSdtEu7XkqYgOw0Nu6HCsb8O9M9Drx31M9IKG9JtqeC/VYj0kzbT8ZSIV
CTv0V4ECgYEA+6pOr+lSfKYg6CVsbniXaba6M7H6iyONZdAP7mmiy2frMakM587V
IrRKv/Sqw6Gqa/EI0MqWfyXuj75Pep3k4GdS+UmbLyQ9quCt/VVoS0ZNIGk9oWWF
1+y6QkoxH+Ua+/w1g5mKunhHaqOQxLR4gcfXhZSHsa2WwhrcIfagoCcCgYEAwSug
hBlPHIytFBdujY3oJqpDEosLNY9pTvzckqThraEaKAZ40xd01I3hlB+HiWEh8v6L
k1XNLCumPjq82Jr0zUZcKsY0glwEGPj9+1zvVxleGAYQjomcbH8nptTEGuRhjNRN
P72nbDHb5HtxjQbLLbZRkkI/vzCFYM0qKZWG3jkCgYEAqZl+aeJtqn8ykgAWcORn
9eAk4A3DcN1GvyRWeC9MfJPN1qu8fMPQsxCZh39VTX23ogLQFBVTEuD3QnvYiCO8
SNaY/lLyd/SOcofz+VQDcycvdvY1iH0ok/l/FIl3Ta1SYBr53XQuH7hLOLx+JXIR
/DQ3NFiCkeUQQkNjsqfIW+UCgYEAhTri1dli4A0CntXecRFL6ccKxbJSw9kqx8jN
FPNVl9yCXtK/6M/Hy4Yj7Hamk5EDykXX3IGUJf/hBBCdCUQuzZnWncXgl3yBcvvg
qMqbZNZhOG6OYtD0qzw0C/O5YdbclTYkJW9YFt54jCi704m4/sk9j4PT1bjQdUhr
6QxxXbkCgYBDA5UHGr5SG/vkaxtZ2wV4KEPXV6Yp8Vny2xmUrIAVFes+EkM0oQey
9r2Ud8L1uGTyeP1mSXr0jpdZHuFoO2uZ/NP6fI+Fqys8AZEVoMpnbTy8OKG16bFj
GnJpvKzHt0NbneG2M1tJYTURpsfp0/8nrOVlzo9YqbAdvS2rsjO+Vg==
-----END RSA PRIVATE KEY-----
matifi@Mac AWS % touch filekey
matifi@Mac AWS % nano filekey
```

Connect the private key

```
ec2-user@10.0.3.24: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[ec2-user@ip-10-0-1-245 ~]$ chmod 400 filekey
[ec2-user@ip-10-0-1-245 ~]$ ssh -i "filekey" ec2-user@10.0.3.24
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-3-24 ~]$ ping google.com
PING google.com (216.58.207.206) 56(84) bytes of data.
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=1 ttl=118 time=4.86 ms
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=2 ttl=118 time=4.69 ms
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=3 ttl=118 time=4.73 ms
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=4 ttl=118 time=4.68 ms
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=5 ttl=118 time=4.76 ms
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=6 ttl=118 time=4.91 ms
64 bytes from arn11s04-in-f14.1e100.net (216.58.207.206): icmp_seq=7 ttl=118 time=4.74 ms
```

Confirm Nat is working in private EC2

```
ermissions 0644 for 'filekey' are too open.
t is required that your private key files are NOT accessible by others.
his private key will be ignored.
oad key "filekey": bad permissions
c2-user@10.0.3.24: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
ec2-user@ip-10-0-1-245 ~]$ chmod 400 filekey
ec2-user@ip-10-0-1-245 ~]$ ssh -i "filekey" ec2-user@10.0.3.24
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
ec2-user@ip-10-0-3-24 ~]$
```